

Practical Homework P4: random-200 KB

[Joelle Dizon]

The Team:

- Joelle Dizon - jkd227@lehigh.edu
- Rachel McIntosh - rem526@lehigh.edu
- Zoe Ford - zmf227@lehigh.edu

Step #1: Importing necessary modules.

The block below imports a necessary module and checks if you are in the right directory. If you do not see a list of python files when you run the block, please make sure you have uploaded the zip file and unzipped it.

In [24]: `import matplotlib.pyplot as plt`

```
%pwd
%ls
```

```
all_facts.txt
atomgenerator.py
autoencoder.py
basictypes.py
chainbased.py
'CSE 327 P4 200-7.ipynb'
'CSE 327 P4 200-8-Copy1.ipynb'
'CSE 327 P4.ipynb'
embedmodel.py
evaluate.py
gameofthrones.txt
gen_queries.py
guided_loss-unification46-110-3-50.png
helpers/
kbencoder.py
kbparser.py
knowledgebase.py
ming-46-110-3-50.csv
mr_back_reasoner.py
mr_train_examples.csv
nnreasoner.py
nnunifier.py
potter_model.pt
potter.txt
__pycache__/
reasoner.py
rKB_model.pth
size200/
std-46-110-3-50.csv
termwalk.py
test_queries.txt
training_and_validation_loss.png
training_loss-10-100-4-50.png
training_loss-10-97-5-50.png
training_loss-45-106-2-50.png
training_loss-46-110-3-50.png
train_queries.txt
triplets.csv
vocab.pkl
vocab.py
```

Step #2: Create a random KB

Creates the files:

- randomKB.txt - A random KB generated using 10 predicates and 100 constants.

- vocab.pkl - Indexes the full set of predicates, constants and variables used by the knowledge base.

In [2]: `%cd size200 # enter correct directory`

```
!python ../../kbencoder.py --generate_kb --num_rules 200 --new_vocab --save_
!head randomKB.txt
```

```
/data/users/jupyter-jkd227/cse327-p4-files/size200
Classes: 10000000.0
Max arity: 3
KB generated (200)
p8(a1).
p5(a19).
p1(a35, a28, a13).
p2(a0).
p5(a11).
p4(a15, a0, a27).
p2(a19).
p9(a69).
p6(a49, a64).
p1(a30, a65, a30).
```

Step #3: Train the neural net

Train a neural net to convert first-order logic atoms into meaningful numeric vectors. We call these vectors "embeddings," and the learned model the embedding model. The code creates random atoms and then uses a mechanism known as triplet loss to place unifying atoms closer together.

Creates the files:

- rKB_model.pth - Saves the learned weights for the embedding model
- training_loss-45-106-2-50.png - A plot of the training loss after each epoch. This file's name depends on the vocabulary of your KB: the four numbers are number of predicates, number of constants, max arity of the predicates, and the embedding size used. If you changed the KB, you'll need to look up the name in your drive and update the code block that displays this file.

In [26]: `%cd size200`

```
!python ../../kbencoder.py --kb_path randomKB.txt --train_unification_model

# for modified embed size:
# !python ../../kbencoder.py --kb_path randomKB.txt --train_unification_model -
```

```
[Errno 2] No such file or directory: 'size200'
/data/users/jupyter-jkd227/cse327-p4-files/size200
Using cuda device
Uni model embed size: 25
[████████████████████] 3500/3500 Anchors generated
[████████████████████] 70000/70000 Triplets generated
[████████████████████] 24298/70000 Triplet encodings
```

```
IOPub data rate exceeded.  
The Jupyter server will temporarily stop sending output  
to the client in order to avoid crashing it.  
To change this limit, set the config variable  
`--ServerApp.iopub_data_rate_limit`.
```

```
Current values:  
ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)  
ServerApp.rate_limit_window=3.0 (secs)
```

```
[██████████] 66358/70000 Triplet encodings
```

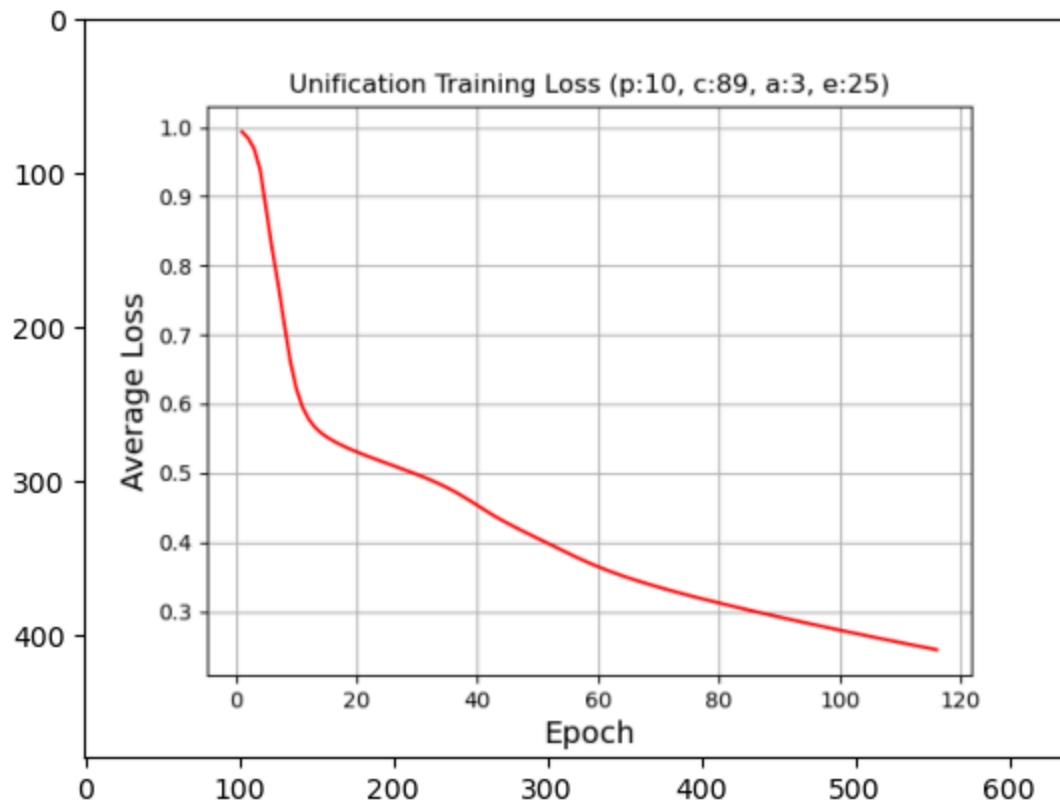
```
IOPub data rate exceeded.  
The Jupyter server will temporarily stop sending output  
to the client in order to avoid crashing it.  
To change this limit, set the config variable  
`--ServerApp.iopub_data_rate_limit`.
```

```
Current values:  
ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)  
ServerApp.rate_limit_window=3.0 (secs)
```

```
Early stopping triggered-----] 23.0% Epoch | [Epoch 116/500, Trainin  
g Loss: 0.2447, Validation Loss: 0.4518]  
Saved model.  
Saved plot.  
unification model generated
```

```
In [27]: # Reading the plot of the training loss  
img = plt.imread('training_loss-10-89-3-25.png')  
plt.imshow(img)
```

```
Out[27]: <matplotlib.image.AxesImage at 0x700a95ab05d0>
```



Step #4: Generate set of train & test queries

Generate a set of train and test queries from your knowledge base. This will also generate the deductive closure (all facts that can be inferred) of your KB using forward-chaining.

Produces the files:

- all_facts.txt - A list of all facts that can be inferred from the KB. This is used to generate queries that are guaranteed to have an answer.
- train_queries.txt - A set of training queries where each will have at least one answer.
- test_queries.txt - A set of test queries where each will have at least one answer.

```
In [8]: !python ../kbencoder.py --kb_path randomKB.txt -g
```

```
Generating facts list...
Forward chaining depth: 1
New facts: 45
Forward chaining depth: 2
New facts: 13
Forward chaining depth: 3
New facts: 2
Forward chaining depth: 4

Facts list generated
60 total facts. Max depth = 3

Number of queries: 200
100 training queries generated
100 testing queries generated
```

Step #5: Create a training set for the scoring model

Create a training set for the scoring model. The examples will consist of goal, rule, score pairs, where a score of 1 means the rule can solve the goal, and 0 means it cannot. It runs the full set of training queries with a randomized backward-chaining process. This step can sometimes take a long time and is not able to take advantage of a GPU (with the gameofthrones KB it took ~45 minutes).

Creates a file:

- `mr_train_examples.csv` - Contains a line for each goal/rule/score example.

```
In [9]: !python ../kbencoder.py --kb_path randomKB.txt -p
```

running training queries...

Negative facts will be generated

1: [p6(X8, X8)]

min depth: 9

min depth: 8

min depth: 7

Nodes: 5937

Answers: 88

2: [p0(X6, a2, X6)]

min depth: 4

Nodes: 17548

Answers: 512

3: [p3(X1, X0), p4(a38, a95, X0), p0(X0, a2, X0)]

min depth: 1

Mismatched predicates: p4(a38, a95, X0) with p1(a20, X0, X1)

Nodes: 5109

Answers: 308

4: [p3(X0, X8)]

min depth: 1

Nodes: 1672

Answers: 150

5: [p4(X1, a41, X0), p1(a10, a1, X1)]

min depth: 1

Nodes: 2

Answers: 1

6: [p6(X9, X9)]

min depth: 9

min depth: 8

min depth: 5

Nodes: 22661

Answers: 601

7: [p1(a88, a33, X0)]

min depth: 1

Nodes: 2

Answers: 1

8: [p4(X1, X0, a43), p5(X0), p0(a90, X1, a16), p0(a22, X0, a0), p0(X0, a74, a0)]

min depth: 1

Nodes: 2

Answers: 1

9: [p4(a73, a30, X0), p5(X0), p0(a37, X0, X1), p6(a56, X1)]

min depth: 1

Nodes: 2

Answers: 1

10: [p2(X3)]

min depth: 2

Nodes: 2738

Answers: 382

11: [p4(X0, a29, a20), p6(X0, X0)]

min depth: 1

Nodes: 2

Answers: 1

12: [p6(X4, X4)]

min depth: 2

Nodes: 10000

Answers: 168

13: [p0(X2, X6, X2)]

min depth: 7

min depth: 4

Nodes: 20902

Answers: 71

14: [p6(X2, X2)]

min depth: 7

Nodes: 18313

Answers: 19

15: [p7(X0), p2(X0), p4(a0, X0, a1), p3(a67, X0)]

min depth: 1

Nodes: 677

Answers: 99

16: [p6(X5, X5)]

min depth: 2

Nodes: 2569

Answers: 92

17: [p8(X0), p1(a35, X0, a13), p1(X0, a62, X1), p0(X2, X1, a17), p4(a0, a1, X2)]

min depth: 1

Nodes: 16

Answers: 15

18: [p0(X7, X4, X7)]

min depth: 4

Nodes: 17195

Answers: 81

19: [p2(X8)]

min depth: 1

Nodes: 454

Answers: 123

20: [p6(X3, X3)]

min depth: 8

min depth: 7

min depth: 2

Nodes: 5360

Answers: 147

21: [p9(X0), p2(X0), p4(X0, a16, a14)]
min depth: 1
Nodes: 15
Answers: 14

22: [p6(X9, X9)]
min depth: 9
min depth: 8
min depth: 7
min depth: 2
Nodes: 13176
Answers: 356

23: [p2(a9)]
min depth: 2
Nodes: 40
Answers: 5

24: [p3(X0, a0)]
min depth: 1
Nodes: 7
Answers: 5

25: [p0(X5, X1, X5)]
min depth: 9
min depth: 8
min depth: 6
min depth: 5
Nodes: 8091
Answers: 553

26: [p3(X0, a53), p1(X1, a62, X0), p7(X1)]
min depth: 1
Nodes: 3
Answers: 1

27: [p3(X8, X5)]
min depth: 1
Nodes: 2072
Answers: 142

28: [p6(a19, X0)]
min depth: 3
Nodes: 985
Answers: 25

29: [p2(X0), p6(X0, X1), p0(a0, a17, X1)]
min depth: 1
Nodes: 2533
Answers: 84

30: [p6(X0, X0), p6(X0, X0)]
min depth: 7
min depth: 2
Nodes: 14017
Answers: 345

31: [p6(X5, X7)]
min depth: 1
Nodes: 22410
Answers: 456

32: [p7(X8)]
min depth: 1
Nodes: 2434
Answers: 179

33: [p2(X2)]
min depth: 1
Nodes: 2362
Answers: 71

34: [p7(X2)]
min depth: 1
Nodes: 11815
Answers: 498

35: [p7(X0), p4(a0, X0, a10), p6(X0, a61)]
min depth: 1
Nodes: 3866
Answers: 101

36: [p3(X1, X2)]
min depth: 1
Nodes: 3403
Answers: 48

37: [p4(X0, X1, a0), p0(a75, a36, X0), p6(a1, X1), p6(X1, X2), p5(X2)]
min depth: 1
Nodes: 4
Answers: 3

38: [p3(a50, X0), p1(a75, X0, a14)]
min depth: 1
Nodes: 4
Answers: 1

39: [p6(X5, X5)]
min depth: 2
Nodes: 13989
Answers: 145

40: [p6(X9, a42)]
min depth: 3
Nodes: 11300
Answers: 1

41: [p2(X0), p0(a23, X1, X0), p0(X2, a61, X1), p6(X2, X2)]
min depth: 1
Nodes: 1347
Answers: 248

42: [p2(X6)]
min depth: 2
Nodes: 7931
Answers: 70

43: [p7(a88)]
min depth: 5
Nodes: 6568
Answers: 27

44: [p6(X7, X7)]
min depth: 8
min depth: 7
min depth: 2
Nodes: 5440
Answers: 105

45: [p0(X8, X5, X8)]
min depth: 4
Nodes: 18731
Answers: 340

46: [p6(X0, X0)]
min depth: 2
Nodes: 21003
Answers: 435

47: [p0(X1, X8, X1)]
min depth: 8
min depth: 5
Nodes: 37327
Answers: 121

48: [p7(X6)]
min depth: 1
Nodes: 2249
Answers: 226

49: [p6(X4, X4)]
min depth: 2
Nodes: 1796
Answers: 80

50: [p2(X1)]
min depth: 1
Nodes: 1648
Answers: 647

51: [p9(X0), p2(X0)]
min depth: 1
Nodes: 15
Answers: 14

52: [p7(a88)]
min depth: 5
Nodes: 116

Answers: 15

53: [p0(X1, X0, X1), p0(a24, X0, a24), p2(X1)]

min depth: 4

Nodes: 11573

Answers: 278

54: [p3(X9, X3)]

min depth: 1

Nodes: 5054

Answers: 418

55: [p6(a19, X7)]

min depth: 3

Nodes: 2138

Answers: 21

56: [p3(X1, X6)]

min depth: 1

Nodes: 3577

Answers: 341

57: [p3(X0, a84), p0(a90, a25, X0)]

min depth: 1

Nodes: 18

Answers: 1

58: [p2(X9)]

min depth: 1

Nodes: 7049

Answers: 458

59: [p7(X2)]

min depth: 1

Nodes: 3417

Answers: 144

60: [p1(a55, a17, X0), p4(a0, X0, a11), p3(X0, a65)]

min depth: 1

Nodes: 2

Answers: 1

61: [p6(X9, X9)]

min depth: 7

min depth: 2

Nodes: 3472

Answers: 76

62: [p8(X0)]

min depth: 1

Nodes: 16

Answers: 15

63: [p0(X5, X6, X5)]

min depth: 9

min depth: 6

min depth: 5
Nodes: 53744
Answers: 633

64: [p0(X7, X6, X7)]
min depth: 7
min depth: 4
Nodes: 19213
Answers: 195

65: [p0(a75, a36, X0), p4(X0, a67, a0)]
min depth: 1
Nodes: 2
Answers: 1

66: [p2(X7)]
min depth: 1
Nodes: 3725
Answers: 690

67: [p3(X5, a15)]
min depth: 3
Nodes: 1011
Answers: 67

68: [p1(X0, a11, a71), p4(X0, a41, a6)]
min depth: 1
Nodes: 2
Answers: 1

69: [p6(X8, X8)]
min depth: 8
min depth: 7
min depth: 2
Nodes: 6105
Answers: 120

70: [p1(X1, a13, X0), p4(a0, a96, X0), p5(X1)]
min depth: 1
Nodes: 3
Answers: 2

71: [p6(X6, X6)]
min depth: 2
Nodes: 4586
Answers: 176

72: [p2(X1)]
min depth: 1
Nodes: 2947
Answers: 35

73: [p6(X0, X1)]
min depth: 2
Nodes: 6224
Answers: 123

74: [p1(X0, a53, X0), p4(a15, X0, a27)]
min depth: 1
Nodes: 2
Answers: 1

75: [p6(a19, X0)]
min depth: 8
min depth: 5
Nodes: 6338
Answers: 216

76: [p0(X4, a2, X4)]
min depth: 5
Nodes: 18515
Answers: 373

77: [p0(X4, X9, X4)]
min depth: 4
Nodes: 2288
Answers: 17

78: [p0(X0, a2, X0)]
min depth: 4
Nodes: 5005
Answers: 136

79: [p3(X0, a19), p1(X0, X1, a76), p1(X1, a65, X1), p4(a73, X1, a4)]
min depth: 1
Nodes: 3
Answers: 1

80: [p8(X0), p3(X0, X1), p0(a45, X1, a0), p4(X0, a34, a79)]
min depth: 1
Nodes: 16
Answers: 15

81: [p6(X4, X4)]
min depth: 2
Nodes: 14560
Answers: 251

82: [p6(a45, a45)]
min depth: 2
Nodes: 13977
Answers: 63

83: [p3(X0, a14), p4(X1, X0, a0), p4(a45, X1, a20)]
min depth: 1
Mismatched predicates: p4(X1, X0, a0) with p1(a20, a14, X0)
Nodes: 3
Answers: 1

84: [p6(X0, X1), p6(X0, X0), p6(X1, X1), p0(X1, a2, X1)]
min depth: 1
Nodes: 16671

Answers: 392

85: [p6(a49, X0), p0(a0, a17, X0), p2(X0)]

Restart...

Restart...

Answers: 0

86: [p0(X0, a4, a55), p6(X0, X0)]

min depth: 1

Nodes: 2

Answers: 1

87: [p6(X0, X1), p6(X0, X0), p4(X1, a96, a11)]

min depth: 1

Nodes: 13026

Answers: 259

88: [p1(X1, a30, X0), p1(a74, X0, a88), p3(X1, a19), p4(a18, a34, X1)]

min depth: 1

Nodes: 2

Answers: 1

89: [p3(X1, X0), p7(X0), p0(a90, a25, X1)]

min depth: 1

Nodes: 3650

Answers: 42

90: [p3(X3, X0)]

min depth: 1

Nodes: 809

Answers: 114

91: [p1(a79, a30, X0), p1(a74, X0, a88)]

min depth: 1

Nodes: 2

Answers: 1

92: [p6(a19, X8)]

min depth: 9

min depth: 6

min depth: 5

Nodes: 7172

Answers: 211

93: [p0(X0, X1, a29), p6(X0, X0)]

min depth: 1

Nodes: 125

Answers: 1

94: [p4(a25, a24, X0), p1(X0, X1, a24), p9(X1)]

min depth: 1

Nodes: 2

Answers: 1

95: [p1(a66, X0, a41), p1(X1, X0, a49), p8(X1), p1(a35, X1, a13), p4(X0, a23, a14)]

min depth: 1
Nodes: 2
Answers: 1

96: [p7(X4)]
min depth: 1
Nodes: 2229
Answers: 86

97: [p6(X6, X6)]
min depth: 2
Nodes: 4328
Answers: 160

98: [p3(X5, X0)]
min depth: 1
Nodes: 1508
Answers: 91

99: [p6(X6, X6)]
min depth: 9
min depth: 7
min depth: 2
Nodes: 10443
Answers: 270

100: [p3(X1, X0), p6(X0, a33), p3(a52, X1)]
min depth: 1
Nodes: 3266
Answers: 330

Before removing duplicates: 377078 examples
After removing duplicates: 4124 examples
pos len=2585, neg len=2585

Step #6: Train the scoring model

Train the scoring model. This step will usually take several minutes, especially if you are not using a GPU (for the gameofthrones KB, it takes about 30 minutes on CPU).

Creates a file:

- uni_mr_model.pt - Contains the weights for the learned scoring model

```
In [32]: !python ../nnreasoner.py --embed_type unification -s random200_reg_model.pt  
  
# for modified nnreasoner:  
# !python ../nnreasoner-mod.py --embed_type unification -s random200_embed_n
```

States from vocab: vocab
Num predicates: 10
Max arity: 3
Num constants: 89
Num variables: 10

Training unification

Embed size: 25

Reading examples from mr_train_examples.csv

Parsing examples...

Generating embeddings...

Embedding input size: 307

[██████████] 5169/5169 Prepping examples

Loaded data...

Using cuda device

/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.

warnings.warn('Lazy modules are a new feature under heavy development ')

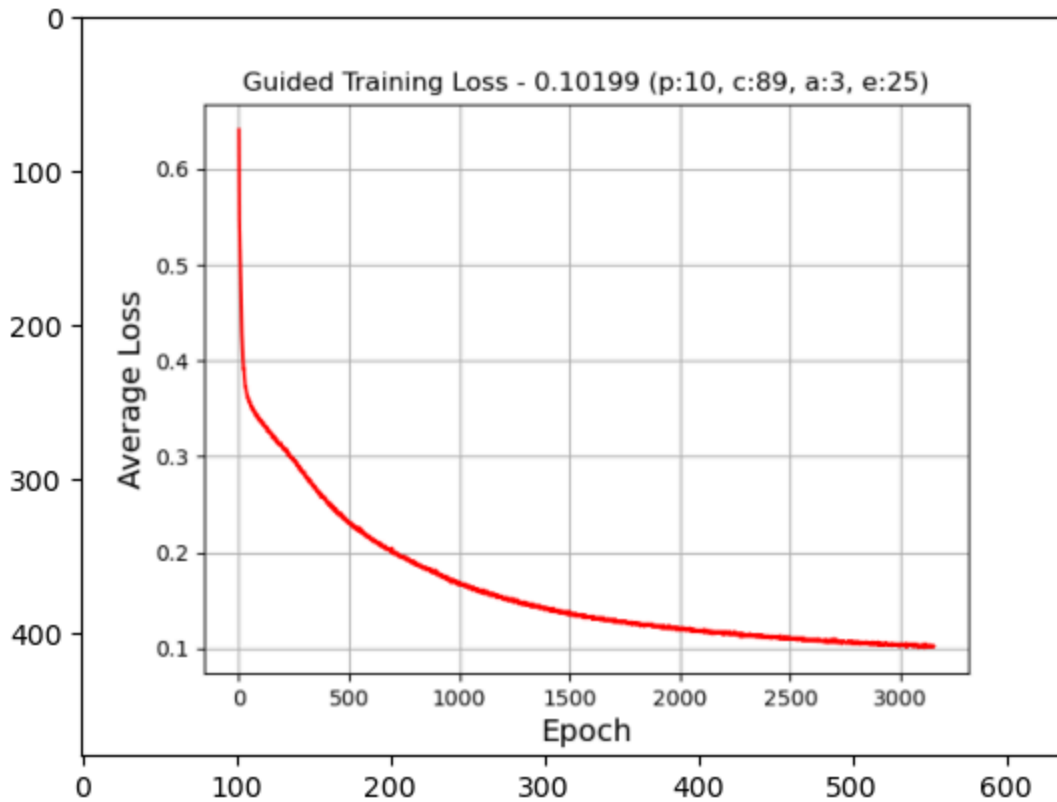
0	0.64116	(-)
50	0.35650	(-0.0054)
100	0.33833	(-0.0029)
150	0.32250	(-0.0020)
200	0.30764	(-0.0016)
250	0.29645	(-0.0013)
300	0.27943	(-0.0012)
350	0.26590	(-0.0010)
400	0.25181	(-0.0009)
450	0.24077	(-0.0008)
500	0.23110	(-0.0008)
550	0.22346	(-0.0007)
600	0.21351	(-0.0007)
650	0.20608	(-0.0006)
700	0.20107	(-0.0006)
750	0.19427	(-0.0005)
800	0.18894	(-0.0005)
850	0.18301	(-0.0005)
900	0.17750	(-0.0005)
950	0.17238	(-0.0004)
1000	0.16689	(-0.0004)
1050	0.16259	(-0.0004)
1100	0.15896	(-0.0004)
1150	0.15608	(-0.0004)
1200	0.15266	(-0.0004)
1250	0.14788	(-0.0003)
1300	0.14586	(-0.0003)
1350	0.14281	(-0.0003)
1400	0.13933	(-0.0003)
1450	0.13942	(-0.0003)
1500	0.13466	(-0.0003)
1550	0.13388	(-0.0003)
1600	0.13152	(-0.0003)
1650	0.13135	(-0.0003)
1700	0.12893	(-0.0003)
1750	0.12804	(-0.0002)
1800	0.12590	(-0.0002)

1850	0.12454	(-0.0002)
1900	0.12304	(-0.0002)
1950	0.12178	(-0.0002)
2000	0.12030	(-0.0002)
2050	0.11996	(-0.0002)
2100	0.11707	(-0.0002)
2150	0.11639	(-0.0002)
2200	0.11449	(-0.0002)
2250	0.11417	(-0.0002)
2300	0.11360	(-0.0002)
2350	0.11197	(-0.0002)
2400	0.11216	(-0.0002)
2450	0.11041	(-0.0002)
2500	0.11030	(-0.0002)
2550	0.10884	(-0.0002)
2600	0.10797	(-0.0002)
2650	0.10678	(-0.0002)
2700	0.10736	(-0.0001)
2750	0.10521	(-0.0001)
2800	0.10509	(-0.0001)
2850	0.10417	(-0.0001)
2900	0.10467	(-0.0001)
2950	0.10399	(-0.0001)
3000	0.10293	(-0.0001)
3050	0.10390	(-0.0001)
3100	0.10336	(-0.0001)
3149	0.10199	(-0.0001)

Saved training loss figure.

```
In [34]: img = plt.imread('guided_loss-unification10-89-3-25.png')  
         plt.imshow(img)
```

```
Out[34]: <matplotlib.image.AxesImage at 0x700a9509ce50>
```



Step #7: Evaluate the scoring model -- compare std (-s) with scoring (-u)

Evaluate the scoring model by running `evaluate.py`. We compare the standard backward-chaining algorithm (-s) with the one that uses our learned scoring model (-u).

The output lists each query, and then provides a line that gives information about query execution. These are: number of nodes explored in the tree, depth of the solution, execution time in seconds, and the speed of exploring the tree (nodes per second).

The names of the files produced depend on the KB used and the options selected.

The code below produces the files:

- `std-45-106-2-50.csv` - A CSV file with query results for the standard backward-chaining algorithm
- `ming-45-106-2-50.csv` - A CSV file with query results for the learned reasoner. `ming` stands for "min scoring goal" and is the default strategy for deciding how to make use of the scores

If you use a different knowledge base, you will likely get different file names that depend on the features of that KB's vocabulary.

```
In [43]: !python ../evaluate.py --kb randomKB.txt -s
!python ../evaluate.py --kb randomKB.txt -u --scoring_model_path random200_r
```

```
# for modified embed size:  
# !python ../evaluate.py --kb randomKB.txt -u --scoring_model_path random200
```

using cuda device
STANDARD

Query 1: [p7(X2)]

2 :: 1 – 00:00:00 (6857 nps)

Query 2: [p3(X0, a15), p4(X0, a1, a10), p3(a48, X0), p1(a26, X0, X1), p1(a80, a52, X1)]

8 :: 7 – 00:00:00 (5700 nps)

Query 3: [p5(X0), p1(a3, X0, X1), p3(X1, a15)]

19 :: 5 – 00:00:00 (5207 nps)

Query 4: [p3(X9, a15)]

4 :: 3 – 00:00:00 (8179 nps)

Query 5: [p7(X5)]

2 :: 1 – 00:00:00 (10767 nps)

Query 6: [p6(a0, X0), p8(X0)]

4 :: 2 – 00:00:00 (8184 nps)

Query 7: [p6(a1, X0), p6(a15, X0), p4(a95, a16, X0), p1(a88, X0, a32)]

6 :: 4 – 00:00:00 (3840 nps)

Query 8: [p3(X3, a15)]

4 :: 3 – 00:00:00 (9206 nps)

Query 9: [p3(X1, X0), p4(X0, X2, a10), p3(X1, X2), p6(X2, a61)]

6 :: 4 – 00:00:00 (5516 nps)

Query 10: [p6(a19, X3)]

6 :: 3 – 00:00:00 (8313 nps)

Query 11: [p6(X3, X3)]

8 :: 7 – 00:00:00 (7244 nps)

Query 12: [p6(X3, X3)]

8 :: 7 – 00:00:00 (82720 nps)

Query 13: [p4(X1, a67, X0), p6(X0, X0), p4(a45, X1, a20), p4(X0, a12, X2), p1(a81, a13, X2)]

7 :: 6 – 00:00:00 (5047 nps)

Query 14: [p6(X2, X2)]

8 :: 7 – 00:00:00 (21222 nps)

Query 15: [p2(X6)]

2 :: 1 – 00:00:00 (7652 nps)

Query 16: [p9(X0), p4(X0, X1, a4), p1(X2, X1, a76), p3(X2, a19)]

10 :: 4 – 00:00:00 (5813 nps)

Query 17: [p2(a61)]

21 :: 15 – 00:00:00 (5806 nps)

Query 18: [p3(X8, X0)]
2 :: 1 – 00:00:00 (6927 nps)

Query 19: [p7(X5)]
2 :: 1 – 00:00:00 (45836 nps)

Query 20: [p2(X0), p1(X1, X2, X0), p7(X1), p1(X3, X2, a49), p8(X3)]
516 :: 5 – 00:00:00 (8361 nps)

Query 21: [p1(X1, X0, a77), p6(X0, a67), p7(X1)]
5 :: 4 – 00:00:00 (7084 nps)

Query 22: [p4(X0, a16, a14), p9(X0)]
3 :: 2 – 00:00:00 (9518 nps)

Query 23: [p7(X1)]
2 :: 1 – 00:00:00 (10963 nps)

Query 24: [p3(X0, a0)]
2 :: 1 – 00:00:00 (9746 nps)

Query 25: [p6(X0, X0), p6(X0, X0), p0(X0, a85, a30)]
4650 :: 6 – 00:00:00 (13147 nps)

Query 26: [p7(X4)]
2 :: 1 – 00:00:00 (10564 nps)

Query 27: [p2(a8)]
21 :: 15 – 00:00:00 (6956 nps)

Query 28: [p2(a38)]
21 :: 15 – 00:00:00 (7019 nps)

Query 29: [p6(X7, X7)]
8 :: 7 – 00:00:00 (17759 nps)

Query 30: [p6(X6, X6)]
8 :: 7 – 00:00:00 (22550 nps)

Query 31: [p6(X8, X7)]
2 :: 1 – 00:00:00 (5225 nps)

Query 32: [p3(X8, X4)]
2 :: 1 – 00:00:00 (5074 nps)

Query 33: [p1(a26, a0, X0), p3(a29, X0)]
3 :: 2 – 00:00:00 (7164 nps)

Query 34: [p7(X0), p6(X0, X0), p6(X0, X0), p0(X0, X1, a27), p8(X1)]
10855 :: 7 – 00:00:00 (32287 nps)

Query 35: [p3(a19, X0)]
4 :: 3 – 00:00:00 (8073 nps)

Query 36: [p7(X1)]
2 :: 1 – 00:00:00 (42064 nps)

Query 37: [p6(X0, a11), p1(X0, a13, X1), p0(X0, a2, X0), p0(a24, X1, X0), p6(X0, X0)]
10 :: 9 – 00:00:00 (10893 nps)

Query 38: [p7(X0), p3(X1, X0), p0(a90, a25, X1), p4(X2, X1, a14), p2(X2)]
24 :: 15 – 00:00:00 (6380 nps)

Query 39: [p3(X7, X1)]
2 :: 1 – 00:00:00 (6902 nps)

Query 40: [p2(X9)]
2 :: 1 – 00:00:00 (7587 nps)

Query 41: [p6(X9, X3)]
2 :: 1 – 00:00:00 (5703 nps)

Query 42: [p2(X0)]
2 :: 1 – 00:00:00 (45593 nps)

Query 43: [p6(X4, X4)]
8 :: 7 – 00:00:00 (22663 nps)

Query 44: [p3(a57, X6)]
4 :: 3 – 00:00:00 (7391 nps)

Query 45: [p5(X0), p1(X0, a87, a24), p6(a22, X0)]
12 :: 3 – 00:00:00 (3461 nps)

Query 46: [p6(X5, X5)]
8 :: 7 – 00:00:00 (21553 nps)

Query 47: [p6(X7, X7)]
8 :: 7 – 00:00:00 (85309 nps)

Query 48: [p2(X5)]
2 :: 1 – 00:00:00 (7522 nps)

Query 49: [p6(a0, a0)]
3 :: 2 – 00:00:00 (5511 nps)

Query 50: [p0(a20, X1, X0), p6(X0, X0), p6(X0, X1), p6(X1, X1)]
14 :: 13 – 00:00:00 (8668 nps)

Query 51: [p4(X1, a95, X0), p0(a45, a50, X0), p1(a40, X1, X0)]
4 :: 3 – 00:00:00 (5931 nps)

Query 52: [p5(X0), p1(a3, X0, a88), p1(X0, a13, a11)]
17 :: 3 – 00:00:00 (6280 nps)

Query 53: [p0(X7, X9, X7)]
105 :: 9 – 00:00:00 (21128 nps)

Query 54: [p0(a45, a85, a30)]
3 :: 2 – 00:00:00 (7577 nps)

Query 55: [p0(X0, X1, X0), p3(X0, a15), p6(X1, X1)]
110 :: 14 – 00:00:00 (55032 nps)

Query 56: [p1(X0, a28, X1), p3(X0, a1), p3(X0, a0), p1(a0, X1, a74)]
5 :: 4 – 00:00:00 (6403 nps)

Query 57: [p3(X8, X9)]
2 :: 1 – 00:00:00 (7179 nps)

Query 58: [p6(X9, X9)]
8 :: 7 – 00:00:00 (22332 nps)

Query 59: [p3(X7, X9)]
2 :: 1 – 00:00:00 (4788 nps)

Query 60: [p6(X4, X4)]
8 :: 7 – 00:00:00 (80095 nps)

Query 61: [p0(a90, X7, a90)]
5 :: 4 – 00:00:00 (5511 nps)

Query 62: [p0(X8, a2, X8)]
10 :: 9 – 00:00:00 (16661 nps)

Query 63: [p6(a24, a24)]
3 :: 2 – 00:00:00 (5842 nps)

Query 64: [p4(X0, X2, X1), p6(a67, X0), p4(a18, X1, a79), p6(X2, a43)]
13 :: 4 – 00:00:00 (5452 nps)

Query 65: [p7(X3)]
2 :: 1 – 00:00:00 (10241 nps)

Query 66: [p7(X2)]
2 :: 1 – 00:00:00 (45944 nps)

Query 67: [p6(a19, X9)]
6 :: 3 – 00:00:00 (8573 nps)

Query 68: [p6(X1, X1)]
8 :: 7 – 00:00:00 (22037 nps)

Query 69: [p4(X1, a12, X0), p3(X0, a1), p2(X1)]
6 :: 3 – 00:00:00 (6063 nps)

Query 70: [p3(a0, X2)]
4 :: 3 – 00:00:00 (9206 nps)

Query 71: [p2(X2)]
2 :: 1 – 00:00:00 (7814 nps)

Query 72: [p0(X2, X5, a30)]
73 :: 2 – 00:00:00 (8904 nps)

Query 73: [p3(X2, X6)]
2 :: 1 – 00:00:00 (6394 nps)

Query 74: [p6(X4, X4)]
8 :: 7 - 00:00:00 (77072 nps)

Query 75: [p3(X2, X0), p3(X0, X1), p6(a19, X1), p2(X2)]
2644 :: 6 - 00:00:00 (10205 nps)

Query 76: [p1(X1, a38, X0), p0(X0, a2, X0), p2(X1), p6(X0, a68)]
8 :: 7 - 00:00:00 (10239 nps)

Query 77: [p3(X8, X5)]
2 :: 1 - 00:00:00 (6642 nps)

Query 78: [p6(X1, X4)]
2 :: 1 - 00:00:00 (580 nps)

Query 79: [p6(X0, X0), p0(a26, X1, X0), p1(a3, X1, a12), p4(a45, a29, X0)]
195 :: 5 - 00:00:00 (33413 nps)

Query 80: [p6(X1, X1)]
8 :: 7 - 00:00:00 (70567 nps)

Query 81: [p7(X1)]
2 :: 1 - 00:00:00 (46249 nps)

Query 82: [p6(X2, X2)]
8 :: 7 - 00:00:00 (90207 nps)

Query 83: [p2(X7)]
2 :: 1 - 00:00:00 (7449 nps)

Query 84: [p0(a45, a50, X0), p1(a47, X0, a57)]
3 :: 2 - 00:00:00 (5527 nps)

Query 85: [p8(X0)]
2 :: 1 - 00:00:00 (12175 nps)

Query 86: [p1(a79, X0, X2), p0(a45, X1, X0), p0(a19, X1, a63), p1(a74, X2, a88)]
6 :: 5 - 00:00:00 (5832 nps)

Query 87: [p6(X1, X1)]
8 :: 7 - 00:00:00 (78570 nps)

Query 88: [p6(X6, X6)]
8 :: 7 - 00:00:00 (82621 nps)

Query 89: [p0(X1, X3, X1)]
105 :: 9 - 00:00:00 (63643 nps)

Query 90: [p6(X1, X0), p3(a31, X0), p0(a20, X1, a19), p4(X1, X2, a1), p2(X2)]
49 :: 5 - 00:00:00 (5856 nps)

Query 91: [p2(X0), p3(a96, X0)]
41 :: 2 - 00:00:00 (6173 nps)

Query 92: [p6(X0, X0), p0(X0, a82, X1), p6(a45, X1), p0(X1, a0, a19), p4(a45, a29, X1)]
204 :: 6 – 00:00:00 (24336 nps)

Query 93: [p6(X2, X2)]
8 :: 7 – 00:00:00 (70444 nps)

Query 94: [p6(X0, a99)]
2 :: 1 – 00:00:00 (7860 nps)

Query 95: [p3(X0, X1), p1(X1, a13, a74)]
4 :: 2 – 00:00:00 (6175 nps)

Query 96: [p3(X1, X8)]
2 :: 1 – 00:00:00 (6892 nps)

Query 97: [p3(X0, X2)]
2 :: 1 – 00:00:00 (7132 nps)

Query 98: [p6(a19, X6)]
6 :: 3 – 00:00:00 (10669 nps)

Query 99: [p6(X0, a74), p0(a75, X1, X0)]
3 :: 2 – 00:00:00 (6642 nps)

Query 100: [p7(a88)]
6 :: 5 – 00:00:00 (11960 nps)

std: 200.74 nodes/query
Time to run all queries: 1.122524948000005
using cuda device
UNITY: ming
 Embedding Model: rKB_model.pth
 Scoring Model: random200_embed_model.pt

Control: mingoal
Rule Eval: Max Rule
Learned Embedding: Network using cuda
/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.
 warnings.warn('Lazy modules are a new feature under heavy development ')

Query 1: [p7(X2)]
2 :: 1 – 00:00:00 (4 nps)

Query 2: [p3(X0, a15), p4(X0, a1, a10), p3(a48, X0), p1(a26, X0, X1), p1(a80, a52, X1)]
8 :: 7 – 00:00:00 (92 nps)

Query 3: [p5(X0), p1(a3, X0, X1), p3(X1, a15)]
1635 :: 5 – 00:00:00 (4436 nps)

Query 4: [p3(X9, a15)]
4 :: 3 – 00:00:00 (69 nps)

Query 5: [p7(X5)]

2 :: 1 – 00:00:00 (95 nps)

Query 6: [p6(a0, X0), p8(X0)]

5 :: 2 – 00:00:00 (226 nps)

Query 7: [p6(a1, X0), p6(a15, X0), p4(a95, a16, X0), p1(a88, X0, a32)]

5 :: 4 – 00:00:00 (232 nps)

Query 8: [p3(X3, a15)]

4 :: 3 – 00:00:00 (70 nps)

Query 9: [p3(X1, X0), p4(X0, X2, a10), p3(X1, X2), p6(X2, a61)]

5 :: 4 – 00:00:00 (56 nps)

Query 10: [p6(a19, X3)]

4 :: 3 – 00:00:00 (92 nps)

Query 11: [p6(X3, X3)]

3 :: 2 – 00:00:00 (90 nps)

Query 12: [p6(X3, X3)]

3 :: 2 – 00:00:00 (26788 nps)

Query 13: [p4(X1, a67, X0), p6(X0, X0), p4(a45, X1, a20), p4(X0, a12, X2), p1(a81, a13, X2)]

7 :: 6 – 00:00:00 (164 nps)

Query 14: [p6(X2, X2)]

7 :: 6 – 00:00:00 (187 nps)

Query 15: [p2(X6)]

2 :: 1 – 00:00:00 (71 nps)

Query 16: [p9(X0), p4(X0, X1, a4), p1(X2, X1, a76), p3(X2, a19)]

5 :: 4 – 00:00:00 (101 nps)

Query 17: [p2(a61)]

21 :: 15 – 00:00:00 (162 nps)

Query 18: [p3(X8, X0)]

2 :: 1 – 00:00:00 (75 nps)

Query 19: [p7(X5)]

2 :: 1 – 00:00:00 (26094 nps)

Query 20: [p2(X0), p1(X1, X2, X0), p7(X1), p1(X3, X2, a49), p8(X3)]

6 :: 5 – 00:00:00 (34 nps)

Query 21: [p1(X1, X0, a77), p6(X0, a67), p7(X1)]

5 :: 4 – 00:00:00 (102 nps)

Query 22: [p4(X0, a16, a14), p9(X0)]

3 :: 2 – 00:00:00 (1385 nps)

Query 23: [p7(X1)]

2 :: 1 – 00:00:00 (30297 nps)

Query 24: [p3(X0, a0)]

2 :: 1 – 00:00:00 (243 nps)

Query 25: [p6(X0, X0), p6(X0, X0), p0(X0, a85, a30)]

5 :: 4 – 00:00:00 (88 nps)

Query 26: [p7(X4)]

2 :: 1 – 00:00:00 (114 nps)

Query 27: [p2(a8)]

21 :: 15 – 00:00:00 (161 nps)

Query 28: [p2(a38)]

21 :: 15 – 00:00:00 (169 nps)

Query 29: [p6(X7, X7)]

3 :: 2 – 00:00:00 (94 nps)

Query 30: [p6(X6, X6)]

3 :: 2 – 00:00:00 (94 nps)

Query 31: [p6(X8, X7)]

2 :: 1 – 00:00:00 (60 nps)

Query 32: [p3(X8, X4)]

2 :: 1 – 00:00:00 (76 nps)

Query 33: [p1(a26, a0, X0), p3(a29, X0)]

3 :: 2 – 00:00:00 (248 nps)

Query 34: [p7(X0), p6(X0, X0), p6(X0, X0), p0(X0, X1, a27), p8(X1)]

6 :: 5 – 00:00:00 (47 nps)

Query 35: [p3(a19, X0)]

4 :: 3 – 00:00:00 (187 nps)

Query 36: [p7(X1)]

2 :: 1 – 00:00:00 (22826 nps)

Query 37: [p6(X0, a11), p1(X0, a13, X1), p0(X0, a2, X0), p0(a24, X1, X0), p6(X0, X0)]

31918 :: 7 – 00:00:04 (6537 nps)

Query 38: [p7(X0), p3(X1, X0), p0(a90, a25, X1), p4(X2, X1, a14), p2(X2)]

48 :: 15 – 00:00:00 (75 nps)

Query 39: [p3(X7, X1)]

2 :: 1 – 00:00:00 (76 nps)

Query 40: [p2(X9)]

2 :: 1 – 00:00:00 (63 nps)

Query 41: [p6(X9, X3)]

2 :: 1 – 00:00:00 (55 nps)

Query 42: [p2(X0)]
2 :: 1 – 00:00:00 (25193 nps)

Query 43: [p6(X4, X4)]
3 :: 2 – 00:00:00 (77 nps)

Query 44: [p3(a57, X6)]
4 :: 3 – 00:00:00 (122 nps)

Query 45: [p5(X0), p1(X0, a87, a24), p6(a22, X0)]
4 :: 3 – 00:00:00 (347 nps)

Query 46: [p6(X5, X5)]
3 :: 2 – 00:00:00 (78 nps)

Query 47: [p6(X7, X7)]
3 :: 2 – 00:00:00 (30714 nps)

Query 48: [p2(X5)]
2 :: 1 – 00:00:00 (60 nps)

Query 49: [p6(a0, a0)]
3 :: 2 – 00:00:00 (515 nps)

Query 50: [p0(a20, X1, X0), p6(X0, X0), p6(X0, X1), p6(X1, X1)]
13 :: 12 – 00:00:00 (82 nps)

Query 51: [p4(X1, a95, X0), p0(a45, a50, X0), p1(a40, X1, X0)]
4 :: 3 – 00:00:00 (366 nps)

Query 52: [p5(X0), p1(a3, X0, a88), p1(X0, a13, a11)]
4 :: 3 – 00:00:00 (603 nps)

Query 53: [p0(X7, X9, X7)]
2438 :: 4 – 00:00:00 (5658 nps)

Query 54: [p0(a45, a85, a30)]
3 :: 2 – 00:00:00 (2248 nps)

Query 55: [p0(X0, X1, X0), p3(X0, a15), p6(X1, X1)]
19 :: 8 – 00:00:00 (48 nps)

Query 56: [p1(X0, a28, X1), p3(X0, a1), p3(X0, a0), p1(a0, X1, a74)]
5 :: 4 – 00:00:00 (223 nps)

Query 57: [p3(X8, X9)]
2 :: 1 – 00:00:00 (78 nps)

Query 58: [p6(X9, X9)]
3 :: 2 – 00:00:00 (97 nps)

Query 59: [p3(X7, X9)]
2 :: 1 – 00:00:00 (78 nps)

Query 60: [p6(X4, X4)]

3 :: 2 - 00:00:00 (29810 nps)

Query 61: [p0(a90, X7, a90)]

5 :: 4 - 00:00:00 (188 nps)

Query 62: [p0(X8, a2, X8)]

2438 :: 4 - 00:00:00 (12995 nps)

Query 63: [p6(a24, a24)]

3 :: 2 - 00:00:00 (516 nps)

Query 64: [p4(X0, X2, X1), p6(a67, X0), p4(a18, X1, a79), p6(X2, a43)]

5 :: 4 - 00:00:00 (85 nps)

Query 65: [p7(X3)]

2 :: 1 - 00:00:00 (101 nps)

Query 66: [p7(X2)]

2 :: 1 - 00:00:00 (24823 nps)

Query 67: [p6(a19, X9)]

4 :: 3 - 00:00:00 (109 nps)

Query 68: [p6(X1, X1)]

3 :: 2 - 00:00:00 (108 nps)

Query 69: [p4(X1, a12, X0), p3(X0, a1), p2(X1)]

1000 :: 3 - 00:00:00 (3409 nps)

Query 70: [p3(a0, X2)]

4 :: 3 - 00:00:00 (614 nps)

Query 71: [p2(X2)]

2 :: 1 - 00:00:00 (26894 nps)

Query 72: [p0(X2, X5, a30)]

3 :: 2 - 00:00:00 (430 nps)

Query 73: [p3(X2, X6)]

2 :: 1 - 00:00:00 (78 nps)

Query 74: [p6(X4, X4)]

3 :: 2 - 00:00:00 (24924 nps)

Query 75: [p3(X2, X0), p3(X0, X1), p6(a19, X1), p2(X2)]

Query failed!!!

1000000 :: 0 - 00:01:05 (15303 nps)

Query 76: [p1(X1, a38, X0), p0(X0, a2, X0), p2(X1), p6(X0, a68)]

8 :: 7 - 00:00:00 (134 nps)

Query 77: [p3(X8, X5)]

2 :: 1 - 00:00:00 (61 nps)

Query 78: [p6(X1, X4)]

2 :: 1 - 00:00:00 (49 nps)

Query 79: [p6(X0, X0), p0(a26, X1, X0), p1(a3, X1, a12), p4(a45, a29, X0)]
6 :: 5 - 00:00:00 (262 nps)

Query 80: [p6(X1, X1)]
3 :: 2 - 00:00:00 (30178 nps)

Query 81: [p7(X1)]
2 :: 1 - 00:00:00 (32795 nps)

Query 82: [p6(X2, X2)]
7 :: 6 - 00:00:00 (43045 nps)

Query 83: [p2(X7)]
2 :: 1 - 00:00:00 (60 nps)

Query 84: [p0(a45, a50, X0), p1(a47, X0, a57)]
3 :: 2 - 00:00:00 (912 nps)

Query 85: [p8(X0)]
2 :: 1 - 00:00:00 (26832 nps)

Query 86: [p1(a79, X0, X2), p0(a45, X1, X0), p0(a19, X1, a63), p1(a74, X2, a88)]
6 :: 5 - 00:00:00 (96 nps)

Query 87: [p6(X1, X1)]
3 :: 2 - 00:00:00 (24084 nps)

Query 88: [p6(X6, X6)]
3 :: 2 - 00:00:00 (29554 nps)

Query 89: [p0(X1, X3, X1)]
5 :: 4 - 00:00:00 (80 nps)

Query 90: [p6(X1, X0), p3(a31, X0), p0(a20, X1, a19), p4(X1, X2, a1), p2(X2)]
6 :: 5 - 00:00:00 (67 nps)

Query 91: [p2(X0), p3(a96, X0)]
39 :: 2 - 00:00:00 (758 nps)

Query 92: [p6(X0, X0), p0(X0, a82, X1), p6(a45, X1), p0(X1, a0, a19), p4(a45, a29, X1)]
7 :: 6 - 00:00:00 (119 nps)

Query 93: [p6(X2, X2)]
7 :: 6 - 00:00:00 (35053 nps)

Query 94: [p6(X0, a99)]
2 :: 1 - 00:00:00 (325 nps)

Query 95: [p3(X0, X1), p1(X1, a13, a74)]
3 :: 2 - 00:00:00 (319 nps)

Query 96: [p3(X1, X8)]

```
2 :: 1 - 00:00:00 (76 nps)

Query 97: [p3(X0, X2)]
2 :: 1 - 00:00:00 (73 nps)

Query 98: [p6(a19, X6)]
4 :: 3 - 00:00:00 (108 nps)

Query 99: [p6(X0, a74), p0(a75, X1, X0)]
34734 :: 2 - 00:00:02 (11823 nps)

Query 100: [p7(a88)]
5 :: 4 - 00:00:00 (394 nps)

ming: 10746.51 nodes/query
1 queries failed
Time to run all queries: 78.92706429199995
```

Step #8: Summary Statistics

The code below shows the first 10 lines of each query result file. A given line has:

- query sequence number
- configuration name
- number of nodes explored
- depth of solution (0 if no solution found)
- did the system find a solution (true/false)
- time in seconds

You can import these files into a spreadsheet in order to create plots or summary statistics.

```
In [20]: !head std-10-89-3-50.csv
         !head ming-10-89-3-50.csv
```

```
query,std reasoner,std nodes explored,std min depth,success,time
1,std,2,1,True,0.000301337000000067876
2,std,8,7,True,0.0013961970000000434
3,std,19,5,True,0.003586340000000021
4,std,4,3,True,0.000476980000000018226
5,std,2,1,True,0.000178785000000012565
6,std,4,2,True,0.00047663900000003895
7,std,6,4,True,0.00152449800000002073
8,std,4,3,True,0.0004339950000000742
9,std,6,4,True,0.00107298700000005803
query,ming reasoner,ming nodes explored,ming min depth,success,time
1,ming,2,1,True,0.4422921449999997
2,ming,8,7,True,0.04314285299999909
3,ming,1635,5,True,0.24861717799999994
4,ming,17,15,True,0.101716246000000051
5,ming,2,1,True,0.0122103770000000605
6,ming,3,2,True,0.024232662999999377
7,ming,5,4,True,0.02176636899999984
8,ming,17,15,True,0.033599924999998976
9,ming,7,4,True,0.05463066999999988
```